# Growth Projections

*compound growth, runway and when to shard*

Watch the 3-min video

**In one line:** project each resource's peak with compound growth, divide into its safe limit to get the runway, and start each big change (replicas, caching, sharding) one lead time before that runway runs out.

## 1 Why you project growth

Your database runs at 40% CPU today and traffic grows fast. Will it survive next year? If you only notice at 100%, you are months too late: **sharding a database takes a team a quarter or more**. A growth projection is how you know when to *start*, not just when you will break.

## 2 Growth compounds

Growth multiplies; it does not add. Ten percent a month is not 120% a year. It is **$1.1^{12} \approx 3.1\times$** in one year. The yearly figure is the **compound annual growth rate (CAGR)**. Three times a year, kept up for three years, is 3 × 3 × 3 = **27×**.

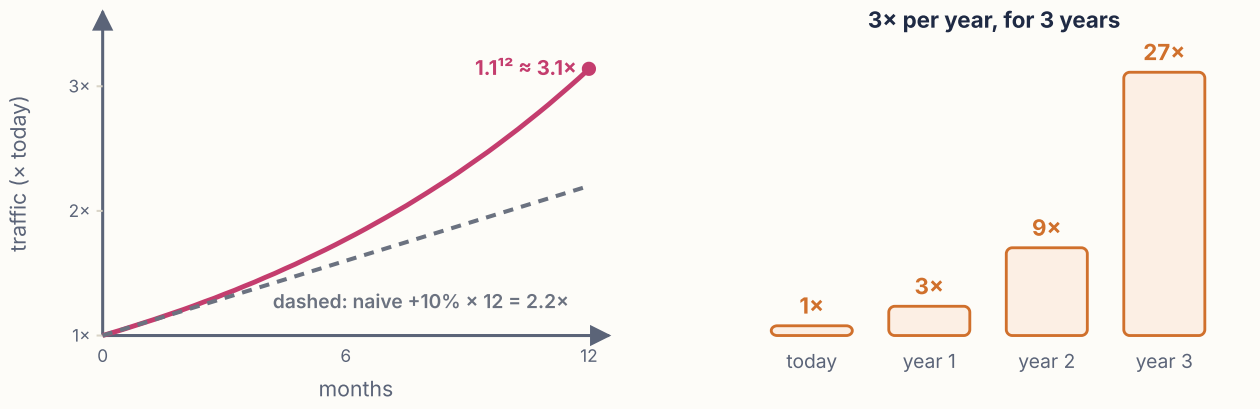



**Figure 1.** Left: 10% a month compounds to about 3.1× in a year, well above the naive 2.2×. Right: 3× a year for three years is 27×.

$$load(t) = load\ today \times (1 + g)^t \quad \cdot \quad monthly\ g = CAGR^{1/12} - 1$$

Going the other way is handy too: "doubles every year" means about **5.9% a month** ($2^{1/12} \approx 1.059$).

## 3 Project per resource, then compute the runway

Start from today's **measured peak**, not the average. Apply the growth rate and calculate the load at future dates: 6 months, 1 year, 2 years, 3 years. Do it **per resource**, because requests per second, storage, database writes and bandwidth often grow at different rates. Storage, for example, only ever accumulates.

Then compare each projection with that resource's capacity and compute the **runway**: how long until it reaches its *safe* limit.

```
runway = log(safe limit ÷ current usage) ÷ log(growth factor per period)
```

Or simply step forward month by month in a spreadsheet. **The resource with the shortest runway is your next bottleneck.**

**WORKED EXAMPLE: THE RUNWAY FORMULA**

A database is at **40% CPU**, traffic grows **10% a month**, and the safe limit is **70%**. Runway = log(70 ÷ 40) ÷ log(1.1) = log(1.75) ÷ log(1.1) ≈ 0.56 ÷ 0.095 ≈ **5.9 months**. Less than half a year, while the fix may take a quarter.

## 4 The workbook case: when to start sharding

The primary database is at **30%** of its write capacity, and writes **double every year**. The trigger for action is **60%**, not 100%, because you need time and headroom for spikes and failover.

- 30% doubles to 60% in exactly **12 months**.
- The sharding project needs **6 months**, so it must start within **6 months**.
- The plan: start the sharding work now, aim to finish before month 12, and treat about **month 18** (≈ 85%, 60% plus a safety buffer) as the absolute latest.

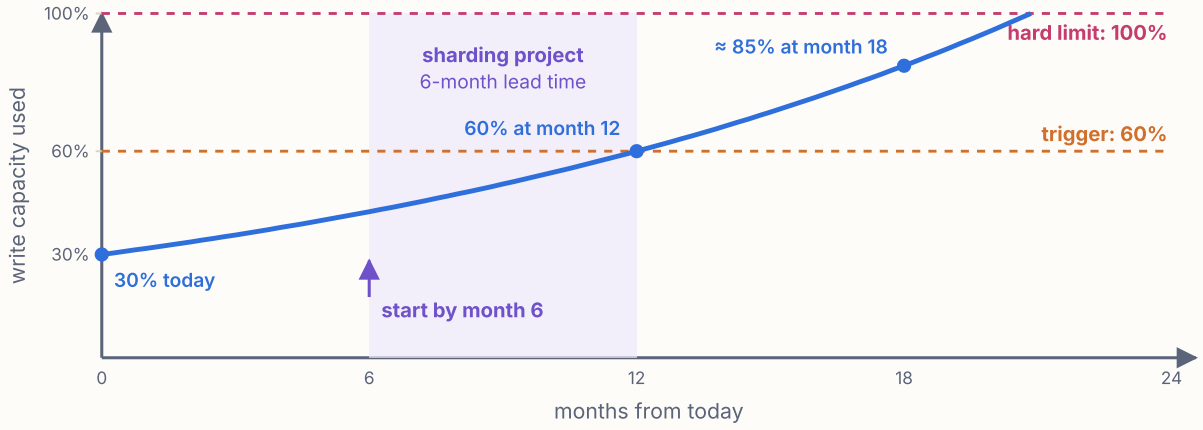



**Figure 2.** Write load doubling yearly from 30%. It crosses the 60% trigger at month 12, so a 6-month project must start by month 6. Waiting for 100% (about month 21) leaves no time.

## 5 Plan re-architecture thresholds in advance

For each limit, write down two things before you need them: a **trigger** (like 60% of capacity) and the **action** it starts. A typical database ladder: add read replicas, scale up, add caching, partition tables, then shard. Each step has a lead time, and the bigger the change, the earlier you must begin.




**Figure 3.** A re-architecture ladder. Lead times are typical ranges, not rules: measure your own team's.

**INTERVIEW ANSWER: PLAN FOR 3× GROWTH NEXT YEAR**

- **Multiply** today's peak for every resource by 3, then add headroom.
- **Check what breaks first.** App servers may just need autoscaling; the database may need replicas or sharding; the cache may need more nodes.
- **Order the work** by runway and lead time: the database usually goes first, because it has the longest lead time.
- **Validate** with load tests at 3× today's traffic.

**COMMON MISTAKES**

- Adding growth linearly: 10% a month is 3.1× a year, not 2.2×.
- Projecting from the average instead of the measured peak.
- Using one growth rate for everything; storage, writes and bandwidth diverge.
- Setting the trigger at 100%. The trigger must leave room for the lead time of the fix.
- Planning only the cheap stateless tier and leaving the database, the slowest change, for last.

## 6 Key terms

**Compound growth**: growth that multiplies each period: $(1 + g)^t$

**CAGR**: compound annual growth rate

**Peak load**: highest measured load, the base for projections

**Capacity**: the most a resource can sustain

**Runway**: time until a resource hits its safe limit

**Trigger threshold**: the level (e.g. 60%) that starts the next step

**Lead time**: how long a change takes from start to ready

**Headroom**: spare capacity kept for spikes and failures

## 7 Interview questions

**Q1. Traffic grows 10% a month. How much bigger is it in a year?** BASIC

$1.1^{12} \approx 3.1\times$, not 2.2×. Growth compounds. In two years it is about 9.8×.

**Q2. What is runway, and how do you compute it?** BASIC

Time until a resource reaches its safe limit. Runway = log(limit ÷ current) ÷ log(growth factor). At 40% CPU, 10% monthly growth and a 70% limit: log 1.75 ÷ log 1.1 ≈ 5.9 months.

**Q3. Why set the trigger at 60% instead of 100%?** BASIC

Big fixes have lead times. Writes doubling yearly go from 60% to 100% in about 9 months; a sharding project can take 6 or more. The gap also absorbs spikes and failover load.

**Q4. The primary is at 30% of write capacity and writes double yearly. Sharding takes 6 months. When do you start?** SCENARIO

30% reaches the 60% trigger at month 12. Start within 6 months, ideally now. By month 18 it is near 85%: the absolute latest.

**Q5. How do you plan for 3× growth next year?** SCENARIO

Multiply each resource's peak by 3 and add headroom. Find what breaks first: app servers autoscale, the database needs replicas or sharding, the cache needs nodes. Order the work by runway and lead time, and load-test at 3×.

**Q6. Requests grow 2× a year but storage grows 4× a year. Which do you watch?** SCENARIO

Both, separately. Storage compounds on top of what you already store, so it can have the shorter runway even though request rates look fine.

**Q7. A doubling-yearly service is at 50% today. Is that safe for 2 years?** TRICKY

No. It reaches 100% at month 12 and 200% at month 24. With a 60% trigger, the runway is only log(1.2) ÷ log(2) ≈ 0.26 years, about 3 months.

**Q8. Why does the database usually go first in a growth plan, even if app servers break earlier?** TRICKY

App servers are stateless: autoscaling them takes minutes. Database changes like sharding take months and move data, so their lead time dominates the schedule.

Next: SD-D-011 · Back-of-the-Envelope Estimation backendroadmap.com